

Getting Started with zeptoforth

[Jump to bottom](#)

tabemann edited this page on Sep 20, 2024 · [51 revisions](#)

Introduction

First, obtain a supported piece of hardware, if you do not have one already. For the purposes of trying out zeptoforth, if you do not already have supported hardware, I would highly suggest getting a Raspberry Pi Pico or Raspberry Pi Pico W — they are quite inexpensive, and are very well-supported by zeptoforth. Note that you will need to solder header pins onto a Raspberry Pi Pico or Raspberry Pi Pico W, unless you buy their counter parts with pre-soldered pins, the Raspberry Pi Pico H and the Raspberry Pi Pico WH, which are at a premium. The largest difference between the Raspberry Pi Pico and the Raspberry Pi Pico W are that the latter has a CYW43439 WiFi (and, not currently supported, Bluetooth) chip while the former does not; this can be controlled through the [CYW43439 firmware](#), and the CYW43439 driver and zeptoIP (which are [here](#)).

If you already have more than one kind of supported hardware, there are trade-offs — RP2040-based boards such as the Raspberry Pi Pico and the Raspberry Pi Pico W supports multiprocessing, unlike the other supported hardware, but has poorer performance due to having Cortex-M0+ cores rather than a Cortex-M4 or Cortex-M7 core. This is due in part to the RP2040's use of external Quad SPI flash combined with a limited-size (16K) XIP flash cache. Also, the RP2040 is a Cortex-M0+ core that is slower clock-for-clock than the more advanced Cortex-M4 or Cortex-M7 cores used by the STM32 platforms that are supported. Conversely, while the RP2350 is more capable than the RP2040, having nearly double the SRAM, more peripherals, and a better Always-On Timer in place of the RTC of the RP2040, the RP2350 is bleeding-edge hardware and has a major silicon bug in the bank 0 GPIO pads in spin A2 regarding pins getting stuck high unless they are pulled down hard by an external source when set to Input Enable which may be fixed in future respins.

Note that support for the Raspberry Pi Pico, the Raspberry Pi Pico W, the Raspberry Pi Pico 2, and the Pimoroni Pico Plus 2 is more complete than for other boards, while at the same time these boards are less expensive than the other supported boards, so it is suggested you get these. (It works on other RP2040 and RP2350 boards, but the `led` module only works "out of the box" on the Raspberry Pi Pico, the Raspberry Pico Plus 2, and the Pimoroni Pico Plus 2 — even though zeptoforth can be configured such that it works on the SeeedStudio Wio RP2040 and XIAO RP2040. It specifically does not work on the Raspberry Pi Pico W, because the LED on it is not controlled directly by the RP2040 but rather is controlled by the CYW43439 WiFi chip, so to control its LED one must use CYW43439 firmware and driver and use those to control the LED.)

Then, build/install zeptoforth and bring up the zeptoforth console on said hardware. Once you have a terminal session with the console up, you should see the following or something similar (it will vary based on what platform you are using, which version of zeptoforth you are using, and when zeptoforth's kernel was compiled):

```
Welcome to zeptoforth
Built for rp2040_big, version 1.2.3, on Sun Oct 29 01:49:46 PM CDT 2023
zeptoforth comes with ABSOLUTELY NO WARRANTY: for details type `license'
ok
```



As zeptoforth is a variety of Forth your normal Forth operations are applicable, such as:

```
.( Hello, World!) Hello, World! ok
1 1 + . 2 ok
:noname 10 0 ?do i . 1000 ms loop ; execute 0 1 2 3 4 5 6 7 8 9 ok
```



For more general guides for beginners on Forth itself, see:

- [Starting Forth](#)
- [A Beginner's Guide to Forth](#)

For a more philosophical text on programming with Forth, see:

- [Thinking Forth](#)

Note however that zeptoforth is not an ANS Forth so not all things mentioned in these texts will transfer directly. For some compatibility words for use with ANS Forth or Forth 200x code, first execute

```
compat import
```



Note that one notable pitfall using zeptoforth compared to some other Forths such as Mecrisp-Stellaris is that in zeptoforth, `variable s` are not automatically initialized in zeptoforth, whether they are compiled to RAM or flash, and hence `variable` does not pop a value off the data stack. Also note that variables compiled to flash only become available after rebooting. When compiling to flash, the recommended approach to initialize variables is to use `init`, as will be seen later.

Basic Peripherals

For some basic things one can do with zeptoforth, you can control LED's (note that this does not apply on RP2040 boards other than the Raspberry Pi Pico, RP2350 boards other than the Raspberry Pi Pico 2 and the Pimoroni Pico Plus 2, or properly-configured Wio RP2040 or XIAO RP2040 boards), as seen with the following:

```
led import  ok
on green led!  ok
off green led!  ok
green toggle-led  ok
green toggle-led  ok
```



This first turns the green LED on, then turns it off, then toggles it on, then toggles it off. Note that on some boards such as the STM32F411 "Black Pill" there is no *green* LED, so one can substitute `green` for, in that case, `blue`, or simply specify `0` in the place of `green` to use the first LED on the board regardless of color.

For a "blinky", i.e. a program to blink an LED on and off, one can use the following:

```
led import  ok
: blinky ( -- )  ok
  begin key? not while green toggle-led 500 ms repeat  ok
  key drop  ok
;  ok
blinky  ok
```



`blinky`, when executed, blinks the green LED on and off, at 500 millisecond intervals, until the user enters a key (note that if the user is using `zeptocom.js` or `e4thcom` keys will not be transmitted until the user hits enter; of course, enter by itself will work here).

There is also a simplified GPIO pin module, `pin`, which enables inputting and outputting digital signals over GPIO pins without having to manually manipulate peripheral registers. Take the following example on the Raspberry Pi Pico where a wire is strung between pins 4 and 5 (GPIO pins 2 and 3):

```
pin import ok
2 output-pin ok
3 input-pin ok
3 pull-up-pin ok
high 2 pin! ok
3 pin@ . -1 ok
low 2 pin! ok
3 pin@ . 0 ok
```



Here GPIO pin 2 is set as an output and GPIO pin 3 is set as a pull-up input. Afterwards, GPIO pin 2 is set high, and a high signal is read from GPIO pin 3. After that, GPIO pin 2 is set low, and a low signal is read from GPIO pin 3. Note that on the Raspberry Pi Pico GPIO pins are simple integers, but GPIO pins are not numbered like the pins on the Raspberry Pi Pico itself (hence why GPIO pin 2 is pin 4 and GPIO pin 3 is pin 5 — pin 3 is a ground, and GPIO pins are counted from 0 while pins are counted from 1).

For a similar example on an STM32 microcontroller using STM32-type peripherals, consider the following example on the STM32L476 DISCOVERY where a wire is strung between PE10 and PE11:

```
pin import ok
10 xe output-pin ok
11 xe input-pin ok
11 xe pull-up-pin ok
high 10 xe pin! ok
11 xe pin@ . -1 ok
low 10 xe pin! ok
11 xe pin@ . 0 ok
```



Here GPIO pin PE10 is set as an output and GPIO pin PE11 is set as a pull-up input. Afterwards, GPIO pin PE10 is set high, and a high signal is read from GPIO pin PE11. After that, GPIO pin PE10 is set low, and a low signal is read from GPIO pin PE11. Note that pins on the STM32 microcontrollers consist of a GPIO and a pin number on that GPIO; `xa` through `xi` (or on the STM32F746 through `xk`) are used to convert pin numbers to pins on the indicated GPIO from GPIOA through GPIOI (or on the STM32F746 through GPIOK). (`xa`, `xb`, `xc`, etc. is used instead of the obvious `pa`, `pb`, `pc`, etc. because `pi` is already taken for 3.14159...)

More Advanced Peripherals

As a bare metal Forth, zeptoforth enables direct access to the hardware from the comfort of the REPL, as shown here on the Raspberry Pi Pico:

```
gpio import ok
25 bit GPIO_OE_SET ! ok
25 bit GPIO_OUT_XOR ! ok
25 bit GPIO_OUT_XOR ! ok
```



In this example we write directly to hardware registers to control the GPIO peripheral, whose registers are defined in the module `gpio`. First we initialize the LED on GPIO pin 25 of the Raspberry Pi Pico (note that this is already initialized on bootup in `full` builds of zeptoforth for the Raspberry Pi Pico, but this is for the purposes of demonstration). Then the state of GPIO pin 25 is toggled twice, turning the LED on and then turning it off again, or vice versa.

For a more advanced example of controlling hardware registers from within zeptoforth (on the Raspberry Pi Pico), consider the following example:

```
gpio import ok
true 3 PADS_BANK0_PUE! ok
false 3 PADS_BANK0_PDE! ok
true 3 PADS_BANK0_IE! ok
true 3 PADS_BANK0_OD! ok
false 2 PADS_BANK0_IE! ok
false 2 PADS_BANK0_OD! ok
2 bit GPIO_OE_SET ! ok
3 bit GPIO_OE_CLR ! ok
2 bit GPIO_OUT_SET ! ok
3 bit GPIO_IN bit@ . -1 ok
2 bit GPIO_OUT_CLR ! ok
3 bit GPIO_IN bit@ . 0 ok
```



Here we configure GPIO pin 3 to be an input with pull-up set and GPIO pin 2 to be an output, and we set GPIO pin 2 first to high and then to low and check the input value of GPIO pin 3. Of course we have a wire connecting pins 4 and 5 (GPIO pins 2 and 3) on the Raspberry Pi Pico to one another so current can be passed from pin 4 to pin 5.

Writing to Flash

To write code that is written to flash, use `compile-to-flash`, as in the following:

```
compile-to-flash ok
: foo ." bar" ; ok
reboot
Welcome to zeptoforth
Built for rp2040, version 0.59.3, on Sun Jan 22 06:15:14 PM CST 2023
zeptoforth comes with ABSOLUTELY NO WARRANTY: for details type `license'
ok
foo bar ok
```



Note that compiling to RAM is the default; to switch to compiling to RAM (e.g. after compiling to flash), use `compile-to-ram`.

To restore zeptoforth to its built image state after having compiled to flash (for a `full` build only), issue the following:

```
restore-state
Welcome to zeptoforth
Built for rp2040_big, version 1.2.3, on Sun Oct 29 01:49:46 PM CDT 2023
zeptoforth comes with ABSOLUTELY NO WARRANTY: for details type `license'
ok
```



To wipe out all compiled code aside from the kernel issue the following:

```
erase-all ok
```



Note that zeptoforth is not very usable after one has done this, as much of the functionality one expects out of a Forth in zeptoforth is written in Forth outside of the zeptoforth kernel. I would only recommend doing this if one intends on rebuilding zeptoforth without rebuilding and reflashing the kernel itself.

Forgetting Words

There are a few different ways of forgetting words, both in RAM and in flash. The typical approaches are to create *cornerstone* and *marker* words with `cornerstone` and `marker`, respectively. `cornerstone` creates a word which, when executed, will erase all words defined after it in either RAM or flash, depending on whether it was originally compiled to RAM or to flash, not including the *cornerstone* word itself. `marker` creates a word which, when executed, will erase all words defined after it along with itself in either RAM or flash, depending on whether it was originally compiled to RAM or to flash.

Be aware that when compiled to flash, `cornerstone` and `marker` take up space so as to align the boundary of the words to be forgotten with a flash erase page boundary, which can potentially waste significant amounts of flash. Also, when *cornerstones* and *markers* compiled to flash are executed the system is rebooted.

For most platforms other than the STM32F411, `full`, `full_swdcom`, and `full_usb` builds come with a `restore-state` cornerstone to return the system to "factory settings" per the build. However, on the STM32F411 there is insufficient space for a cornerstone with `full` and `full_swdcom` builds, and creating a cornerstone or marker with these builds on the STM32F411 will render the system unusable by exhausting all of the flash.

Additionally, there is a word specifically for forgetting arbitrary words compiled to RAM along with every word defined after them named `forget`. This word is a no-op when applied to words compiled to flash.

Take care when forgetting words in RAM, because if any tasks are running which may reference such words, or all hooks are set to refer to such words, undefined behavior will occur once said words are forgotten.

Utilizing PSRAM

Some RP2350 boards such as the Pimoroni Pico Plus 2 come with PSRAM, which is short for Pseudostatic RAM, which is connected to the RP2350 MCU via Quad SPI. On these boards, PSRAM can be initialized by executing `init-psram` (`psram-cs-pin --`), where *psram-cs-pin* is the index of the GPIO tied to the PSRAM's chip select pin. On the Pimoroni Pico Plus 2, this is GPIO 47. Once this is executed, `psram-size` (`-- bytes`) will return the size of the PSRAM in bytes. Also, the base address of the PSRAM can be gotten with `psram-base` (`-- address`).

Note that PSRAM is slower than SRAM, as it communicates with the RP2350 MCU over a Quad SPI link, and it competes with Quad SPI flash for XIP cache space.

If you wish for PSRAM to initialized on bootup, execute the following:

```
compile-to-flash
: do-init-psram <psram-cs-pin> init-psram ;
initializer do-init-psram
reboot
```



where `<psram-cs-pin>` is the index of the GPIO tied to the PSRAM's chip select pin.

Some Basic Developer Tools

To print all words in the current namespace issue `words` , as seen below:



```
words
restore-state      init          init          usb
init              fat32-tools  simple-fat32  init
fat32             sd          block-dev     init
rtc              init        pwm           init
i2c              init        spi           init
adc              init        uart          init
dma-pool          dma         edit          edit-internal
list-range        load-range  list          load
init             block       qspi          rng
init             timer       monitor       tinymt32
action-pool       task-pool   line-enabled? disable-line
enable-line       init        line-internal page
ansi-term         console     stream        rchan
schan            chan        fchan         sema
init             alarm       lock          tqueue
action           core-lock   slock         init
oo              int-map     cstr-map      map
heap            pool        see-for-gas   see
see-xt-for-gas   see-xt      disassemble-for-gas disassemble
disassemble-internal
s\"               c\"
c\"              temp-str
closure          compat
init            ms
task            pio
init           int-io
systick        init
parse-double-unsigned
asinh          tanh
f**            acos
atan           tan
ln            lnp1
factorial     sqrt
d*            dmax
2rot          4dup
[snipped for brevity]
```

To list the words in a specific module (also known as a wordlist), specify the name of the module followed by `words-in` :


```

spi words-in
spi>buffer          buffer>spi          drain-spi          flush-spi
spi>?              >spi?              spi>              >spi
spi-pin            spi-alternate        spi-rx-handler!    disable-spi-loopback
enable-spi-loopback disable-spi-tx          enable-spi-tx        disable-spi
enable-spi          spi-data-size!        natl-microwire-spi  ti-ss-spi
motorola-spi        slave-spi          master-spi          spi-baud!
spi-internal        x-invalid-spi-data-size          x-invalid-spi-clock
x-invalid-spi
ok

```



One may also look up all the words sharing a given prefix with `lookup` followed by the prefix:

```

lookup x-
x-domain-error      x-not-found          x-already-defined    x-order-overflow
x-stack-underflow   x-stack-overflow     x-insufficient-data
x-no-word-being-built
x-not-compiling      x-token-expected     x-failed-parse        x-hook-needed
x-unknown-word        x-domain-error        x-not-found          x-already-defined
x-order-overflow      x-stack-underflow     x-stack-overflow      x-insufficient-data
x-no-word-being-built
x-not-compiling      x-token-expected     x-failed-parse        x-hook-needed
x-unknown-word
ok

```



As with `words` and `words-in`, one may also look up all the words sharing a given prefix in a particular module with the module followed by `lookup-in` followed by the prefix:

```

task lookup-in x-
x-interrupt-main      x-current-wait-notify
x-out-of-range-notify          x-timed-out
x-out-of-range-priority        x-in-task-change      x-would-block
x-terminated
ok

```



To disassemble a selected word (rather than an address range) issue `see` followed by the word, as seen below:

```

see disassemble
10027A5C B500:      disassemble:      PUSH {LR}
10027A5E 3F04:      SUBS R7, #$4
10027A60 603E:      STR R6, [R7]
10027A62 2600:      MOVS R6, #$0
10027A64 481E:      LDR R0, #10027AE0 @ 20010624

```



```

10027A66 6006:          STR R6, [R0]
10027A68 CF40:          LDMIA R7!, {R6}
10027A6A 3F04:          SUBS R7, #$4
10027A6C 603E:          STR R6, [R7]
10027A6E 2600:          MOVS R6, #$0
10027A70 481C:          LDR R0, #$10027AE4 @ 20010728
10027A72 6006:          STR R6, [R0]
10027A74 CF40:          LDMIA R7!, {R6}
10027A76 6838:          LDR R0, [R7]
10027A78 3F08:          SUBS R7, #$8
10027A7A 607E:          STR R6, [R7, #$4]
10027A7C 6038:          STR R0, [R7]
10027A7E 0030:          MOVS R0, R6
10027A80 683E:          LDR R6, [R7]
10027A82 6038:          STR R0, [R7]
10027A84 6838:      3:          LDR R0, [R7]
10027A86 3F08:          SUBS R7, #$8
10027A88 607E:          STR R6, [R7, #$4]
10027A8A 6038:          STR R0, [R7]
10027A8C 0030:          MOVS R0, R6
10027A8E 683E:          LDR R6, [R7]
10027A90 6038:          STR R0, [R7]
10027A92 4915:          LDR R1, #$10027AE8 @ 200023B5
10027A94 4788:          BLX R1
[snipped for brevity]

```

To dump memory as bytes (organized into four-byte groups) along with ASCII, use `dump`, which dumps memory from a starting address up to but not including an ending address, as seen below:

```

' welcome ' welcome 256 + dump
10011234 00 B5 17 49 88 47 04 3F 3E 60 00 00 01 A6 0C E0 |...I.G.?>`.....|
10011244 00 BF FE DE 15 57 65 6C 63 6F 6D 65 20 74 6F 20 |.....Welcome to |
10011254 7A 65 70 74 6F 66 6F 72 74 68 0E 49 88 47 0E 49 |zeptoforth.I.G.I|
10011264 88 47 0E 49 88 47 04 3F 3E 60 00 00 01 A6 07 E0 |.G.I.G.?>`.....|
10011274 00 BF FE DE 0A 42 75 69 6C 74 20 66 6F 72 20 00 |.....Built for .|
10011284 07 49 88 47 07 49 88 47 07 49 88 47 07 49 0F E0 |.I.G.I.G.I.G.I..|
10011294 35 24 00 20 CF 24 00 20 5F 24 00 20 35 24 00 20 |5$. _$. _$. 5$. |
100112A4 CF 24 00 20 5F 24 00 20 C1 1C 00 20 5F 24 00 20 |.$. _$. ... _$. |
100112B4 88 47 04 3F 3E 60 00 00 01 A6 07 E0 00 BF FE DE |.G.?>`.....|
100112C4 0A 2C 20 76 65 72 73 69 6F 6E 20 00 0C 49 88 47 |., version ..I.G|
100112D4 0C 49 88 47 0C 49 88 47 0C 49 88 47 04 3F 3E 60 |.I.G.I.G.I.G.?>`|
100112E4 01 A6 04 E0 00 BF FE DE 05 2C 20 6F 6E 20 08 49 |....., on .I|
100112F4 88 47 08 49 88 47 08 49 88 47 08 49 10 E0 00 00 |.G.I.G.I.G.I....|
10011304 CF 24 00 20 5F 24 00 20 05 1D 00 20 5F 24 00 20 |.$. _$. ... _$. |
10011314 CF 24 00 20 5F 24 00 20 3D 1D 00 20 5F 24 00 20 |.$. _$. =.. _$. |

```



```
10011324 88 47 20 49 88 47 04 3F 3E 60 00 00 01 A6 19 E0 |.G I.G.?>`.....|
ok
```

Numbers

zeptoforth supports parsing numbers with a variety of `base` values (from 2 to 16), which may be set with `binary` (for base 2), `octal` (for base 8), `decimal` (for base 10, the default), and `hex` (for base 16). Each task has its own `base` value. Note that `base` affects both numeric parsing and numeric formatting. Additionally, aside from the value of `base`, and base of a single number may be set with the prefixes `%` (for base 2), `/` (for base 8), `#` (for base 10), and `$` (for base 16), without changing the value of `base`.

It should be noted that zeptoforth has no native support for floating-point math, as not all the supported platforms, i.e. the Raspberry Pi Pico, have hardware floating-point support, and of those that do, the Cortex-M4-based boards, i.e. the STM32F407 and STM32L476 DISCOVERY boards, only have hardware for supporting single-precision floating-point math. However, S31.32 fixed-point math using double cells is supported by zeptoforth. Take the following examples:

```
pi f. 3,1415926499 ok
1,0 exp f. 2,7182818271 ok
pi 2,0 f/ sin f. 1,0000000004 ok
2,0 ln f. 0,6931471810 ok
2,0 ln exp f. 1,9999999993 ok
1,0 3,0 f/ 2dup d+ f. 0,6666666665 ok
4,0 sqrt f. 2,0 ok
1,5 2,0 f* f. 3,0 ok
```



Here we can see that S31.32 fixed-point numbers are expressed with `x,y` where `,` is the decimal point; this is because `x.y` is already traditionally used in Forth for expressing non-fixed-point double cell numbers. Also it can be seen that `d+` is used for not just double-cell addition but also S31.32 fixed-point addition because the two are equivalent; the same is true of `d-` and subtraction. On the other hand, though, there are dedicated `f*` and `f/` words for S31.32 fixed-point multiplication and division, because fixed-point multiplication and division are not interchangeable with non-fixed-point double-cell multiplication and division.

Numbers can have underscores in them at any position after the base prefix and the negative prefix `-`. `1_000_000`, for single-cell `1000000`, `$CAFE_BABE` for single-cell `$CAFEBABE`, `100_200_300.`, for double-cell `100200300`, and `1000_._005` for fixed-point `1000,005` are all valid. However `_-1000` or `_$DEADBEEF` are not valid. The purpose of this is to enable the user to break up numbers for readability purposes.

Printing strings to the console is carried out with the following words in most cases:

`u. ( u -- )`

Print an unsigned single-cell integer.

`. ( n -- )`

Print a signed single-cell integer

`ud. ( ud -- )`

Print an unsigned double-cell integer.

`d. ( nd -- )`

Print a signed double-cell integer.

`f. ( f -- )`

Print an S31.32 fixed-point value. Note that the decimal point is `,`.

`f.n ( f places -- )`

Format an S31.32 fixed-point value in a buffer *truncated* (not rounded) to a given number of places past the decimal. Note that the decimal point is `,`.

All these words use the current `base` value for determining the base of their printed output.

Another thing to consider is formatting numbers as strings. There are the `<#`, `#`, `#s`, `hold`, `#>` words traditionally used by Forth, which I will not discuss here. zeptoforth, however, also provides the following words:

`format-unsigned ( c-addr u -- c-addr bytes )`

Format an unsigned single-cell integer in a buffer.

`format-integer ( c-addr n -- c-addr bytes )`

Format a signed single-cell integer in a buffer.

`format-double-unsigned` (c-addr ud -- c-addr bytes)

Format an unsigned double-cell integer in a buffer.

`format-double` (c-addr nd -- c-addr bytes)

Format a signed double-cell integer in a buffer.

`format-fixed` (c-addr f -- c-addr bytes)

Format an S31.32 fixed-point value in a buffer. Note that the decimal point is `,` .

`format-fixed-truncate` (c-addr f places -- c-addr bytes)

Format an S31.32 fixed-point value in a buffer *truncated* (not rounded) to a given number of places past the decimal. Note that the decimal point is `,` .

Note that for convenience's sake, these words return the buffer address passed in and the total number of bytes the value is formatted as, so the output can be used immediately as a string. These words use the current base when formatting numbers. Also note that they assume that the buffer contains enough space available for any possible number that may be formatted with it. (E.g. with using `format-double-unsigned` with a `base` of 2 one must have 64 bytes of space available in the buffer.)

There are also the following words for parsing numbers from strings:

`parse-unsigned` (c-addr bytes -- u success?)

Parse the contents of a string as an unsigned integer.

`parse-integer` (c-addr bytes -- n success?)

Parse the contents of a string as a signed integer.

`parse-double-unsigned` (c-addr bytes -- ud success?)

Parse the contents of a string as an unsigned double-cell integer.

`parse-double` (c-addr bytes -- nd success?)

Parse the contents of a string as a signed double-cell integer.

`parse-fixed` (c-addr bytes -- f success?)

Parse the contents of a string as a fixed-point number.

All of these words return the parsed value and whether parsing was successful; if parsing was not successful, the parsed value is to be discarded.

Structures

zeptoforth enables the definition of structures, as seen below:

```
begin-structure foobar-size ok
  cfield: foo ok
  hfield: bar ok
  field: baz ok
  2field: qux ok
  16 +field quux ok
end-structure ok
foobar-size buffer: my-foobar ok
0 my-foobar foo c! ok
1 my-foobar bar h! ok
2 my-foobar baz ! ok
3 4 my-foobar qux 2! ok
my-foobar quux 16 5 fill ok
```



Here a structure is defined whose size in bytes is placed in a constant `foobar-size` and which has one byte-sized field `foo`, one halfword-sized field `bar`, one cell-sized field `baz`, one double cell-sized field `qux`, and one 16-byte-sized field `quux`. Alignment of halfword, cell, and double cell-sized fields is automatic; note that double cell-sized fields are cell-aligned, not double-cell aligned. Fields' words simply take their structure's addresses and add their offset to it, returning the address of said field within the structure in question.

Modules

When writing non-trivial code, or code which one otherwise wants to control the namespace usage of, it is very useful to make use of the zeptoforth module system:

```
begin-module foobar ok
  : foo ." F00" ; ok
  : bar ." BAR" ; ok
  : baz ." BAZ" ; ok
end-module ok
foo unable to parse: foo
foobar import ok
foo F00 ok
```



Here a module `foobar` is created, containing the words `foo` , `bar` , and `baz` ; before the words may be used, the containing module must be imported with `import` , or the containing module path must be explicitly referenced with *module* `:: word` as shown below:

```
begin-module quux ok
  begin-module quux-internal ok
    42 constant the-answer ok
  end-module> import ok
  ' the-answer export the-answer ok
end-module ok
the-answer . unable to parse: the-answer
quux :: the-answer . 42 ok
```



Note that `::` is an immediate word which takes a module off the stack so when used within a word definition a *module* must be enclosed in `[` and `]` in most use cases.

Also, here modules are nested, with `quux-internal` being created inside `quux` , `end-module>` is used to end a module's definition while returning the module's ID, which is then passed to `import` to import that module in the containing module, `export` is used to export an execution token from within a module so that it can be used when the module is imported or referenced later.

Note that a common idiom used in zeptoforth when compiling to the `forth` module (i.e. the default module) is to do the following:

```
continue-module forth ok
  led import ok
  : blinky begin green toggle-led 500 ms key? until ; ok
end-module ok
green toggle-led unable to parse: green
blinky ok
```



Here a namespace is defined by `continue-module` , which here specifies that subsequent definitions are to be added to the `forth` module. Once `end-module` is reached any modules imported within the module continuation are unimported, so `green` (and `toggle-led`) is not left in the namespace, but any words defined between `continue-module` and `end-module` are defined in the default module.

Exceptions

For an example of exceptions in zeptoforth, take the following:



```
led import ok
: x-my-exception ." my exception has been raised" cr ; ok
: raise-my-exception ( x -- ) ok
  on green led! [: 1000 ms dup 0<> averts x-my-exception ;] try nip off green led! ?
raise ; ok
1 raise-my-exception ok
0 raise-my-exception my exception has been raised
```

This example defines an exception, `x-my-exception` (by convention all exceptions in zeptoforth begin with `x-`) and defines a word, `raise-my-exception` which takes a value, turns on the LED, waits one second so the user can see the LED's being on, checks whether the value is non-zero, raises `x-my-exception` if the value is zero (`averts` takes the name of an exception and raises the exception if the value on the stack is zero; compare with `triggers`, which takes the name of an exception and raises the exception if the value on the stack is non-zero), catches the exception with `try` (which is passed the execution token of the code the exception to be caught is raised within), turns the LED off whether or not the exception has been raised or not, and then re-raises the exception with `?raise` if it is non-zero (zero as returned by `try` indicates no exception).

Quotations

This example also shows the usage of quotations defined by `[:` and `;`]. Quotations are similar to anonymous words defined by `:noname` and `;` except they are defined within other word definitions, and their execution token is compiled in place as a literal.

Note that `:noname ... ;` cannot be nested inside another word definition and are meant for creating top-level anonymous word definitions (e.g. for use for defining deferred words or methods); bad things will happen if you try to nest it inside another word. Conversely, `[:` and `;`] are meant to be nested inside other word definitions.

Initialization and Turnkey Operation

To initialize state (e.g. variables) on bootup, define the word `init` in flash. Any `init` must, as the first thing it does, call `init` to initialize any other state needed; failing to do so will break zeptoforth, as there is critical setup which is done by preexisting `init` words. Then, in your `init` initialize whatever state you need initialize.

However, if `turnkey` operation is specifically desired, a better approach is to define `turnkey` in flash with it doing everything one wants to do as a turnkey; if one wants to return the user to the REPL, simply return from `turnkey`. Note that `turnkey` is always executed after `init`, even if `init` is defined after `turnkey`. Take the following:

```
compile-to-flash ok
variable counter ok
: init init 0 counter ! ; ok
: turnkey begin counter @ . 1000 ms 1 counter +! key? until ; ok
reboot 0 1 2 3 4 5 6 7 8 9 10
Welcome to zeptoforth
Built for rp2040, version 0.59.3, on Sun Jan 22 06:15:14 PM CST 2023
zeptoforth comes with ABSOLUTELY NO WARRANTY: for details type `license'
ok
```



Here we first initialize on bootup everything else that needs initialization before this point. Afterwards, we initialize the variable `counter` that had been compiled to flash to zero. Finally, with our turnkey defined as `turkney`, we print out the value of `counter`, wait one second, and then increment it until the user presses a key. Once the user presses a key, the welcome message is displayed and the user is presented with the REPL.

Basic Multitasking

For a simple example of multitasking, take the following:

```
task import ok
led import ok
variable blinky-task ok
0 :noname begin green toggle-led 500 ms again ; 256 128 512 spawn blinky-task ! ok
blinky-task @ run ok
blinky-task @ stop ok
```



Here `spawn` creates a blinky task (the numbers are the dictionary size, data stack size, and return stack size in bytes, respectively) and returns it. `blinky-task @ run` starts the blinky, which will blink while the user will have full control of the REPL, until the user executes `blinky-task @ stop`. Note that `blinky-task @ run` and `blinky-task @ stop` can be repeated any number of times, resuming and suspending the blinky task respectively.

For more on multitasking and multiprocessing, see [Multitasking and Multiprocessing with zeptoforth](#).

Find a page or section...

▼ Home

▼ Actions and Action Schedulers in zeptoforth

▼ Bitmaps, Pixmaps, Fonts, and Displays for zeptoforth

▼ Building and Using zeptoIP

▼ Closures in zeptoforth

▼ Console Redirection in zeptoforth

▼ Control Structures in zeptoforth using Quotations

▼ Dynamically-Scoped Variables in zeptoforth

▼ Getting Started with zeptoforth

Introduction

Basic Peripherals

More Advanced Peripherals

Writing to Flash

Forgetting Words

Utilizing PSRAM

Some Basic Developer Tools

Numbers

Structures

Modules

Exceptions

Quotations

Initialization and Turnkey Operation

Basic Multitasking

▼ Implicit Compilation in zeptoforth

▼ Installing and Building zeptoforth and Using the zeptoforth Console

▼ Memory Management with zeptoforth

▼ Modules in zeptoforth

▼ Multitasking and Multiprocessing with zeptoforth

▼ Object Orientation in zeptoforth

Show 8 more pages...

Clone this wiki locally

<https://github.com/tabemann/zeptoforth.wiki.git>

